

AI-Powered Audit Document Tracking System

and Follow-Up

Week 10: Workflow Automation Capstone

Dennis Paul and Associates never misses an overdue audit document again. We engineered a production-grade AI workflow that classifies risk, drafts professional emails, gates them through human approval, and logs every decision automatically.

Submitted by: Leilah Kokoyo and Dennis Paul | Date: June 2026

PROJECT OBJECTIVES

This project was designed to achieve the following specific, measurable outcomes:

- Automate daily identification of overdue audit documents from the client tracking sheet without any manual trigger.
- Classify each overdue record by risk level (Low, Medium, or High) using an AI agent, replacing subjective manual assessment.
- Generate tone-appropriate client reminder emails dynamically using a dedicated AI Email Drafter Agent, replacing three hardcoded static templates.
- Enforce a human approval gate for all Medium and High risk communications before any email reaches a client.
- Log every classification and outcome to Google Sheets automatically, creating a complete audit trail for every workflow execution.
- Deploy a separate AI-powered Error Handler workflow that diagnoses failures and delivers plain-English notifications to the team.
- Produce a system capable of processing ten or more client records per daily run without human intervention in execution.

THE PROBLEM

Now let us consider how to lose audit efficiency right in our face.

Every day Dennis Paul and Associates manages the processing of documentation requests from up to ten clients. In each of those requests there could be urgent ones that can directly affect the audit timeline, yet because all the pending documents were on one and the same Google Sheet, they could easily go unnoticed. When such an overlooked request was discovered, it was too late for a polite follow-up reminder.

There are several ways to solve the problem using automated solutions, yet in the past attempts were always unsuccessful. The reason behind all failures lay within manually performed procedures. Email reminders were pre-set templates with no differentiation between the number of overdue days for a particular client, thus making it impossible to make a risk level classification for the follow-up email. There was also no possibility for the creation of a proper follow-up template, or an opportunity to have another person approve the email before it reaches the customer.

And that is only half of the problem; the other half concerns the construction of a fully-fledged system capable of risk assessment, intelligent email generation, human involvement in the day-to-day execution.

SOLUTION

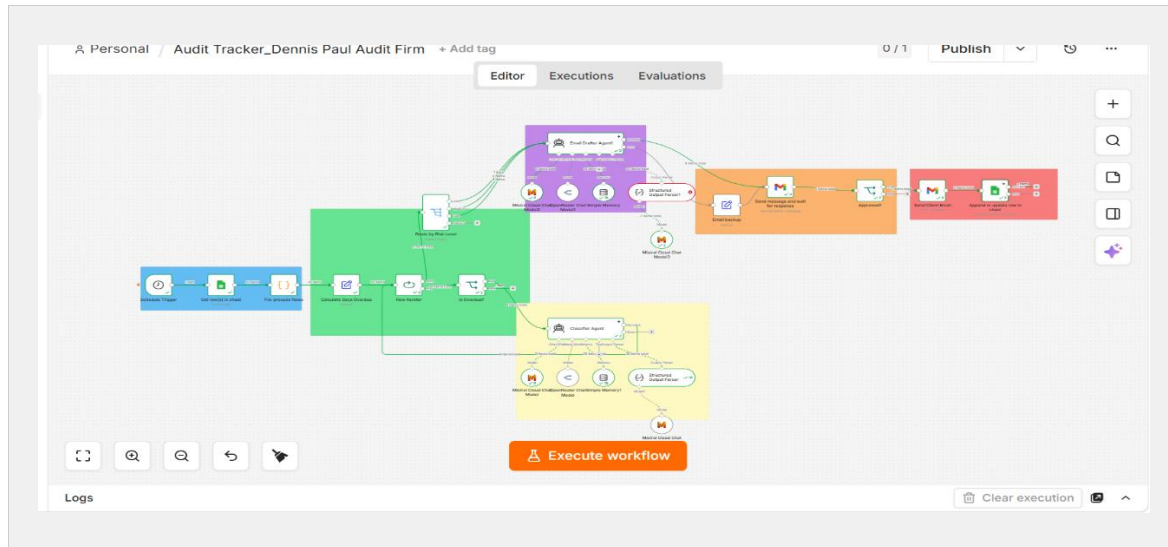
The workflow for the AI-Powered Audit Document Tracking and Follow-Up System was created in n8n. The workflow is fired automatically every day at 8:00 AM. The system reads from the

Audit Tracker Google Sheet data on each document, calculates the number of overdue days, filters documents that are currently overdue, sends each document to be classified by the AI Classifier Agent, drafts tone-appropriate reminder emails using the AI Email Drafter Agent, runs each high/medium-risk email through the manager's approval, then sends approved emails to clients, and finally logs the results into Google Sheets with no need for human interaction throughout the process.

There are five critical engineering decisions that made this workflow production-ready:

- The Pre-process Rows code node that standardizes the date format and sets a unique ID for each row before performing any calculations;
- The Is Overdue IF node filtering out documents that are currently not overdue, saving the costs associated with calling AI services unnecessarily;
- The use of two distinct AI Agents that take responsibility only for classification and draft only emails respectively;
- An approval gate for humans that uses the Wait node in n8n sends an email to the manager containing links to Approve/Reject before sending the email to the client.
- The AI-based Error Handler workflow will catch any failure in the process and send a smart analysis to the team via email.

Figure 1 — Full n8n Canvas — Six colour-coded visual groups from left to right: Data Input (Blue), Loop and Logic (Green), Classifier AI (Yellow), Email Drafter AI (Purple), Approval Flow (Orange), Output (Red).



ARCHITECTURE

System Architecture Overview

This flow is entirely based on n8n Cloud and uses three third-party applications – Google Sheets (as data source and storage), Gmail (for communications), and Mistral AI (through OpenRouter API as the intelligence component). No database is involved in this flow. All the states are stored in Google Sheets.

The data flow is unidirectional and goes from Google Sheets to n8n via two AI-based agents consecutively followed by the approval stage (if needed) and then to Gmail and Google Sheets again. The error handling module operates independently and is notified about any errors occurred in n8n automatically.

Component Diagram

The architecture consists of the following layers:

Layer	Component	Purpose
Data Source	Google Sheets — Audit Tracker	Stores all client records, due dates, status, and workflow outputs
Orchestration	n8n Cloud	Hosts and executes all workflow logic, scheduling, and routing
Intelligence	Mistral AI via OpenRouter	Powers both the Classifier Agent and Email Drafter Agent
Communication	Gmail (OAuth2)	Sends approval requests to manager and reminder emails to clients
Scheduling	n8n Schedule Trigger	Fires the workflow automatically every day at 08:00
Error Handling	Separate n8n workflow	Receives failure events, diagnoses with AI, notifies team via Gmail

Data Flow

- Schedule Trigger fires at 08:00 daily.
- Google Sheets node reads all rows where Status equals Pending.
- Pre-process Rows Code node normalizes dates and assigns row IDs.
- Row Iterator loops one record at a time through the downstream pipeline.
- Calculate Days Overdue computes the delay in days for each record.
- Is Overdue? IF node filters records where Days Pending is zero or less.
- Classifier Agent receives each overdue record and returns a risk classification JSON.

- Route by Risk Level Switch node directs the record to the correct downstream path.
- Email Drafter Agent receives the classified record and drafts a tone-matched email.
- Needs Approval? IF node routes Low risk records to direct send and Medium/High through the approval gate.
- Approval gate pauses execution until the manager clicks Approve or Reject.
- Send Client Email Gmail node delivers the approved email to the client.
- Update Audit Tracker Google Sheets node writes back Risk Level, Days Pending, and Reminder Sent.

INTEGRATIONS, APIS, AND TOOLS USED

Tool / Service	Type	How Used	Authentication
n8n Cloud	Workflow Automation	Hosts all workflow logic, scheduling, agents, and routing	n8n Cloud account
Google Sheets	Data Storage	Source of client audit records; also receives workflow write-backs	OAuth2
Gmail	Email Delivery	Sends approval requests to manager and reminder emails to clients	OAuth2
Mistral AI (OpenRouter)	AI Language Model	Powers Classifier Agent and Email Drafter Agent	OpenRouter API Key
GitHub	Version Control	Stores workflow JSON files and project documentation	Personal Access Token
n8n Wait Node (Webhook)	Human-in-the-Loop	Pauses execution until manager approves or rejects via email link	n8n internal

API Notes

- Mistral AI is accessed via OpenRouter's unified API endpoint. The model used is mistral-medium. Temperature is set to 0.2 for the Classifier Agent and 0.5 for the Email Drafter Agent.
- Google Sheets is connected via OAuth2. The workflow reads from and writes back to the same sheet using row matching on Client Name.
- Gmail is connected via OAuth2. Two separate Gmail nodes are used: one for manager approval requests and one for client emails.
- The n8n Wait node generates a unique resumeUrl per execution. No external webhook service is required.

STEP-BY-STEP WORKFLOW EXPLANATION

STEP 1: DATA INPUT AND PRE-PROCESSING

Schedule Trigger + Get row(s) in sheet + Pre-process Rows

The workflow starts with a Schedule Trigger configured to fire every day at 8:00 AM. No manual run. No human needed to start the process.

Immediately after the trigger, a Google Sheets node named Get row(s) in sheet reads all rows from the Audit Tracker sheet where Status equals Pending. The sheet contains the following columns:

Column	Type	Description
Client ID	String	Unique identifier for each client (e.g. DPA001)
Client Name	String	Name of the audit client
Client Email	String	Email address for reminder delivery
Document Requested	String	Name of the outstanding document
Due Date	Date	Submission deadline for the document
Status	String	Current status: Pending or Submitted
Days Pending	Number	Calculated overdue days (written back by workflow)
Risk Level	String	AI-assigned classification: Low, Medium, or High
Reminder Sent	String	Flag updated to YES after email is sent

After reading the rows, a Code node named Pre-process Rows normalizes the data before it enters any calculation. The node adds two fields to every row:

```
return items.map((item, i) => ({
  ...item,
  json: {
    ...item.json,
    row_id: i + 1,
    normalized_due: new Date(item.json['Due Date']).toISOString()
  }
}));
```

The design principle here is deliberate. Date normalization happens once, at the source, in a dedicated node. Downstream calculations never need to guess whether they are receiving a raw Google Sheets date string or an ISO timestamp. The row_id field ensures each record is uniquely trackable through the rest of the flow.

Figure 2 — Schedule Trigger NODE — Configured to fire at 08:00 every day.

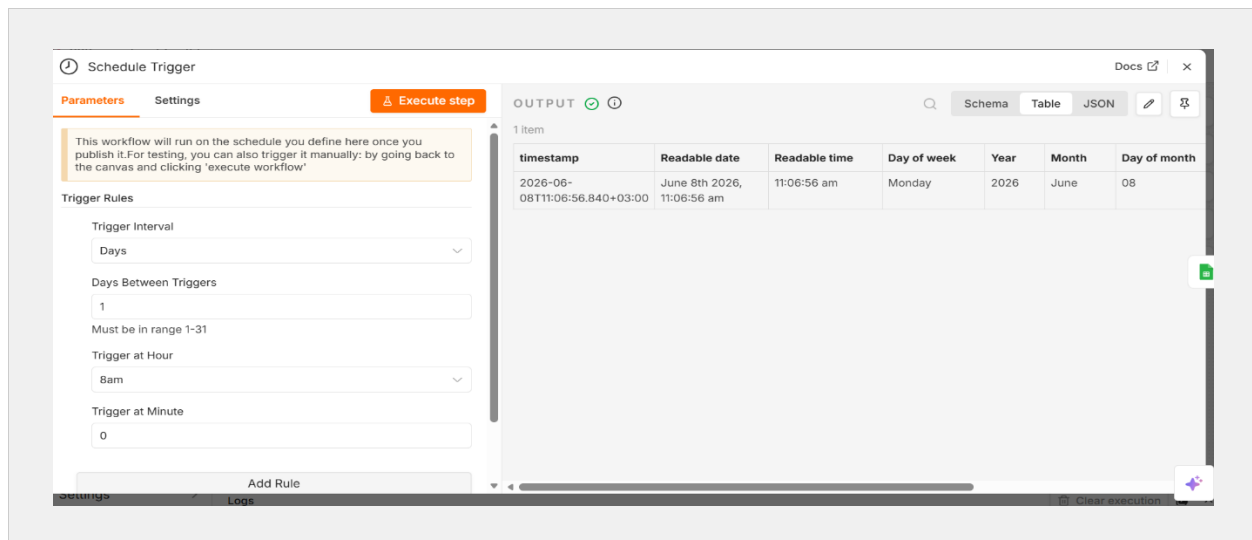


Figure 3 — Pre-process Rows CODE NODE — Normalizes Due Date to ISO format and assigns a row_id to each record.

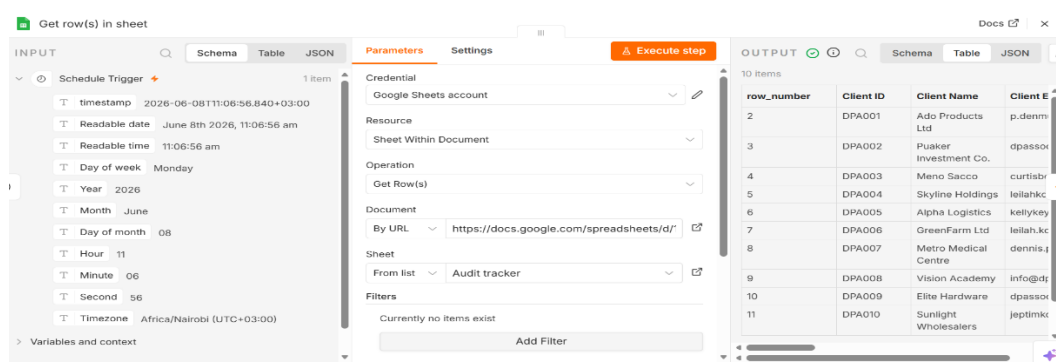
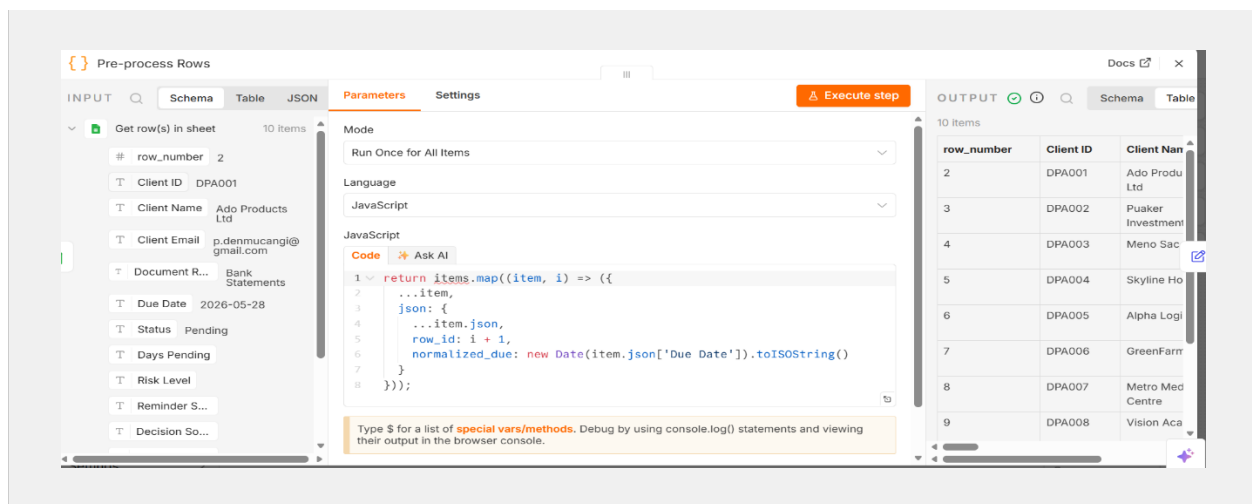
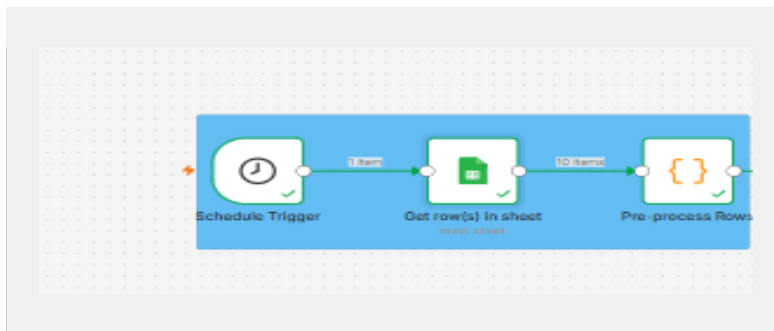


Figure 4 — Data Input Visual Group (BLUE) — Schedule Trigger, Google Sheets, and Pre-process Rows nodes grouped and colour-coded blue.



STEP 2: LOOP AND OVERDUE FILTERING

Row Iterator + Calculate Days Overdue + Is Overdue?

Once preprocessed, the data goes into the Loop Over Items block called the Row Iterator with a batch size of 1. This is the most critical step in this workflow structure-wise. In fact, to avoid the batch mode, this step is mandatory for feeding data into the AI Classifier Agent that has to process clients individually. Each client passes down the entire pipeline, getting classified, drafted, approved, and logged one-by-one, until the next row enters the loop.

From the loop's output, each row goes into the Calculate Days Overdue Set block, where three new variables are calculated from the normalised due date:

Field	Type	Expression
Days Pending	Number	<code>Math.floor((Date.now() - new Date(\$json['Due Date'])).getTime()) / 86400000)</code>
overdue	String	Returns 'true' if Days Pending > 0, 'false' otherwise
days_label	String	Human-readable label: '7 days overdue', 'Due today', '3 days remaining'

The Days Pending field is typed as Number explicitly. This prevents the type mismatch error that caused the original workflow to fail when the IF node attempted a numeric comparison against a string value.

After calculation, each row passes through an IF node named Is Overdue?. This is the scope guard for the AI pipeline. The condition is:

```
Condition: {{ $json['Days Pending'] }} is greater than 0 (type: Number)
```

Rows where Days Pending is zero or negative exit on the False branch and are returned to the Row Iterator without any AI processing. This guard eliminates unnecessary AI API calls and prevents the Classifier Agent from ever receiving an irrelevant row.

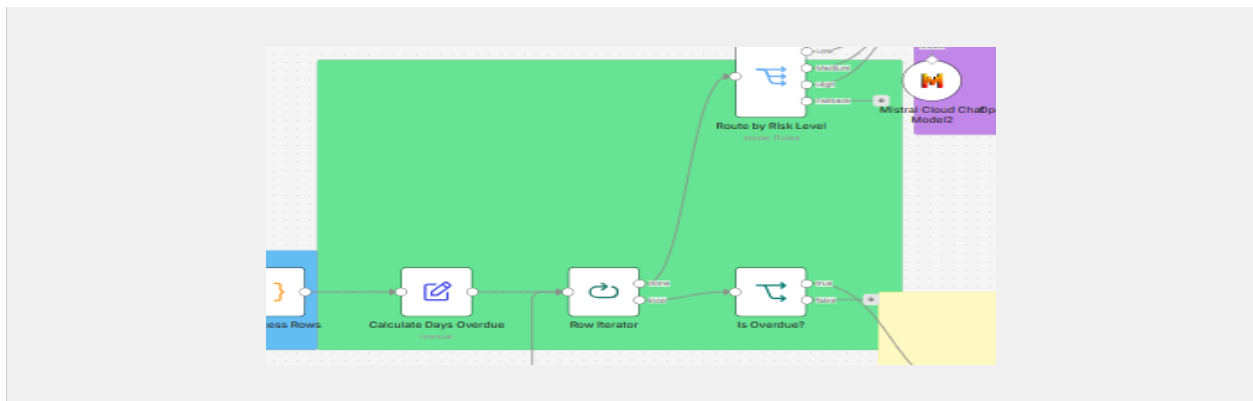
Figure 5 — Row Iterator NODE — Loop Over Items node with batch size 1.

The screenshot shows the 'Row Iterator' node configuration. The 'Parameters' tab is selected, displaying 'Batch Size' as 5. The 'INPUT' section shows a table with 10 items, including fields like 'row_number', 'Client ID', 'Client Name', 'Client Email', 'Document Requested', 'Due Date', 'Status', 'Days Pending', 'Risk Level', 'Reminder Sent', 'Decision Source', and 'Last Reminder Date'. The 'OUTPUT' section shows the results of the iteration, including client details and document information.

Figure 6 — Is Overdue? NODE — IF node filtering non-overdue records before they reach the AI pipeline.

The screenshot shows the 'Is Overdue?' node configuration. The 'Parameters' tab is selected, displaying 'Conditions' as 'Days Pending' is greater than 0. The 'INPUT' section shows a table with 5 items, including fields like 'row_number', 'Client ID', 'Client Name', 'Client Email', 'Document Requested', 'Due Date', 'Status', 'Days Pending', 'Risk Level', 'Reminder Sent', 'Decision Source', and 'Last Reminder Date'. The 'OUTPUT' section shows the results of the iteration, including client details and document information.

Figure 7 — Loop and Logic Visual Group (GREEN) — Row Iterator, Calculate Days Overdue, and Is Overdue? nodes.



STEP 3: INTELLIGENCE LAYER — AI CLASSIFIER AGENT

Classifier Agent + Route by Risk Level + Write to Classifications Tab

Each overdue task that gets through the Is Overdue? filter goes to the first Intelligence Layer. There is a node in this layer called Classifier Agent. This agent is programmed to do only one thing — classify the level of risk for the delay. It does not write the email. It does not decide the tone of the message. Its result comes out in one JSON file.

This agent uses a Mistral Cloud Chat Model where the temperature value is 0.2. This means its results will be focused and consistent. The Structured Output Parser helps enforce the required JSON structure. Auto-Fix is on to correct small discrepancies in the result.

The Classifier Agent node connects to three sub-nodes:

- Mistral Cloud Chat Model — primary language model for risk classification.
- Simple Memory — session memory keyed by timestamp, preventing context bleed between client records.
- Structured Output Parser — enforces the seven-field JSON schema.

After classification, the output fans to two parallel destinations simultaneously:

- Write to Classifications Tab — a Google Sheets Append Row node that logs the classification result to a dedicated Classifications sheet tab.
- Route by Risk Level — a Switch node that routes the classified record to the appropriate downstream branch based on risk_level.

The Switch node contains three rules:

```
Rule 1: {{ $json.output.risk_level }} equals Low    -> Output 0
Rule 2: {{ $json.output.risk_level }} equals Medium -> Output 1
Rule 3: {{ $json.output.risk_level }} equals High   -> Output 2
```

Figure 8 — Classifier Agent NODE — AI Agent classifying each overdue record.

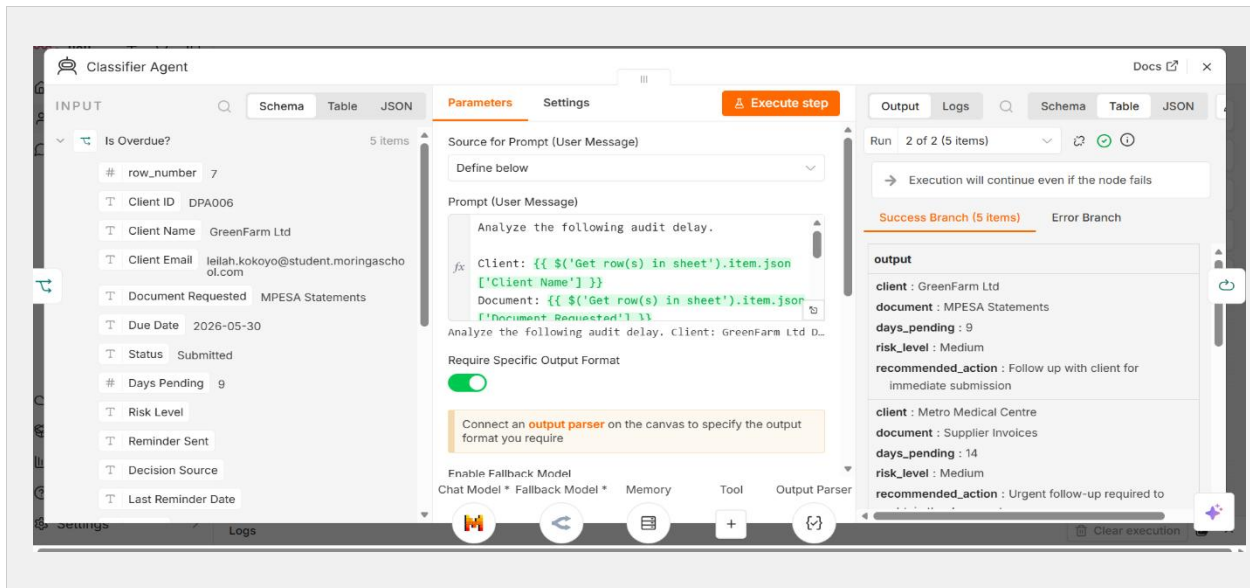


Figure 9 — Structured Output Parser NODE — Enforces the seven-field JSON schema. Auto-Fix enabled.

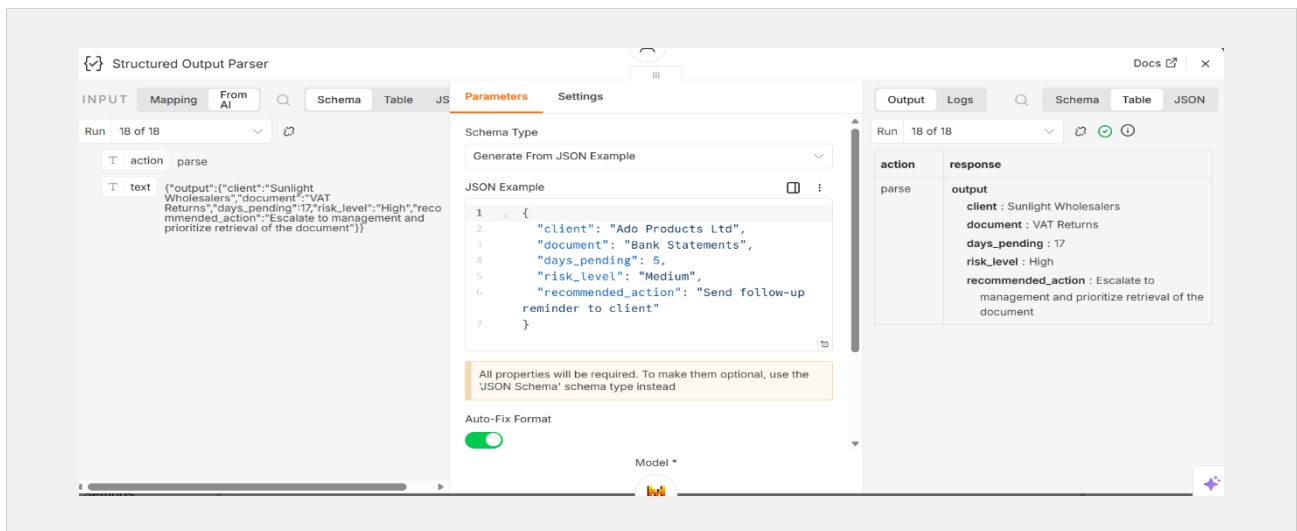


Figure 10 — Route by Risk Level NODE — Switch node routing Low, Medium, and High risk records.

Route by Risk Level

INPUT (5 items):

#	row_number	Client ID	Client Name	Client Email	Document Requested	Due Date	Status	Days Pending	Risk Level	Reminder Sent	Decision Source	Last Reminder Date
7		DPA006	GreenFarm Ltd	leilah.kokoyo@student.moringaschool.com	MPESA Statements	2026-05-30	Submitted	9				

Parameters:

- Mode: Rules
- Routing Rules: `{{ $('Classifier: Agent').item.json.output.risk_level }}`
- Is equal to: Low
- Rename Output: ☒
- Output Name: Low

OUTPUT (Low (1 item), Medium (4 items), High (4 items))

output:

```
{
  "client": "Vision Academy",
  "document": "Cashbook",
  "days_pending": 3,
  "risk_level": "Low",
  "recommended_action": "Monitor and remind client if delay extends beyond 5 days"
}
```

Edit System Message

System Message

You are a risk classification assistant for an audit firm in Kenya.

You receive details about an overdue audit document and you classify the delay risk.

Rules:

1. Low risk: 1 to 5 days overdue
2. Medium risk: 6 to 14 days overdue
3. High risk: 15 or more days overdue
4. If Days Pending is 0 or negative, classify as Low

Return ONLY this exact JSON structure. No explanation. No markdown. No code blocks:

```
{
  "client": "client name here",
  "document": "document name here",
  "days_pending": 0,
  "risk_level": "Low",
  "recommended_action": "brief action here"
}
```

risk_level must be exactly one of: Low, Medium, High

Is Overdue? (5 items)

INPUT (5 items):

#	row_number	Client ID	Client Name	Client Email	Document Requested	Due Date	Status	Days Pending	Risk Level	Reminder Sent
7		DPA006	GreenFarm Ltd	leilah.kokoyo@student.moringaschool.com	MPESA Statements	2026-05-30	Submitted	9		

Expression (Anything inside {{ }} is JavaScript. [Learn more](#))

Analyze the following audit delay.

```
Client: {{ $('Get row(s) in sheet').item.json['Client Name'] }}
Document: {{ $('Get row(s) in sheet').item.json['Document Requested'] }}
Days Pending: {{ $json['Days Pending'] }}
Classify the risk and return ONLY this JSON:
{
  "client": "client name here",
  "document": "document name here",
  "days_pending": 0,
  "risk_level": "Low",
  "recommended_action": "brief action here"
}
```

Result

Item: C

```
Client: GreenFarm Ltd
Document: MPESA Statements
Days Pending: 9
Classify the risk and return ONLY this JSON:
{
  "client": "client name here",
  "document": "document name here",
  "days_pending": 0,
  "risk_level": "Low",
  "recommended_action": "brief action here"
}
```

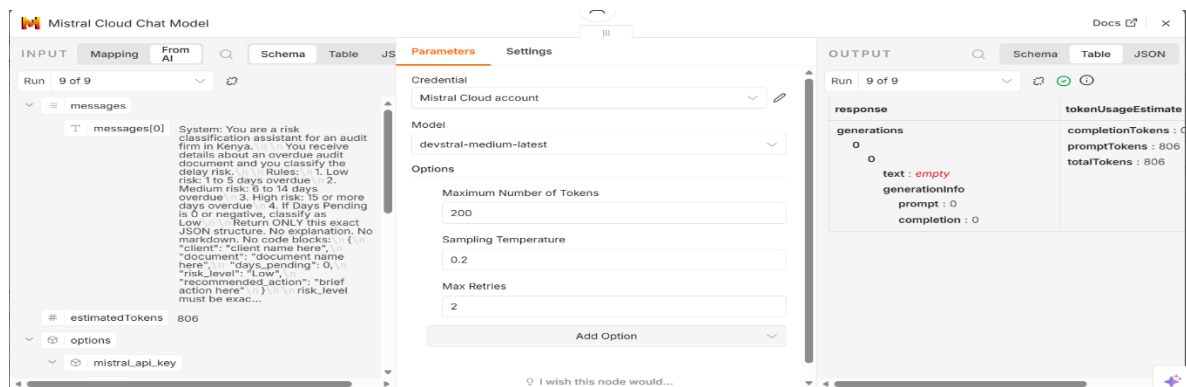
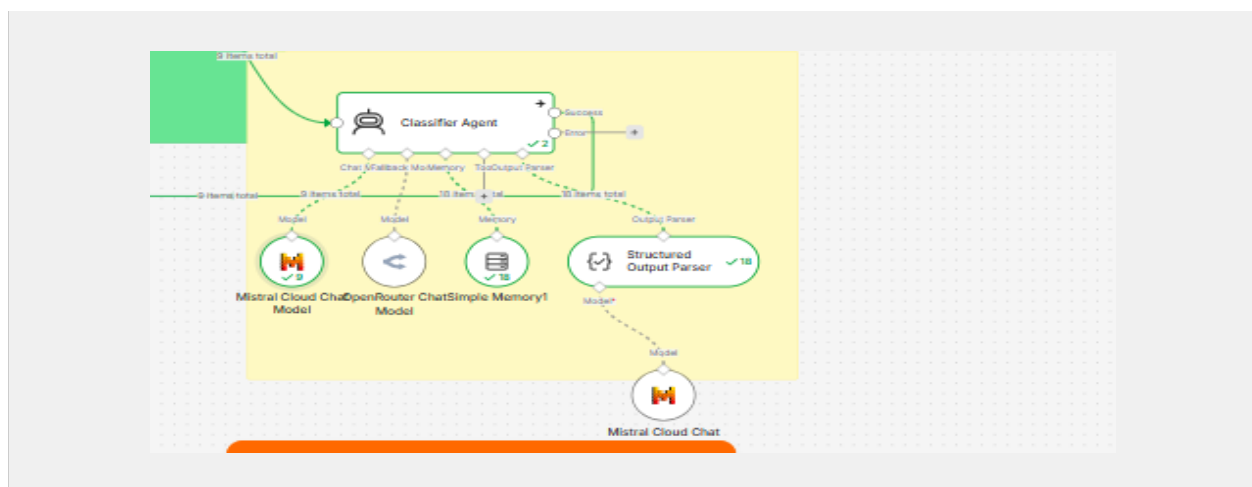


Figure 11 — Classifier AI Visual Group (YELLOW) — Classifier Agent, Route by Risk Level, and Write to Classifications Tab nodes.



STEP 4: INTELLIGENCE LAYER — EMAIL DRAFTER AGENT

Email Drafter Agent

Each record that passes through the Route by Risk Level Switch (notwithstanding which branch the record took) proceeds onto the second tier of intelligence: an AI Agent node called Email Drafter Agent. There is only one job for this agent – draft the client reminder email. It does not re-assess risk. It assesses the risk class from the Classifier Agent to establish tone.

The Low, Medium, and High Switch branches all converge on the single Email Drafter Agent node. The purposeful separation of concern by the two agents was considered carefully. The Classifier Agent makes the analysis call while the Email Drafter Agent communicates the message. In the case of an independent update to either Agent node, nothing would break.

The output is accessed downstream as `$json.output.body` via a Structured Output Parser attached to this agent. This clean reference replaces all three hardcoded Gmail template bodies from the original workflow. One AI agent, writing dynamically, replaces three static templates.

Figure 12 — Email Drafter Agent NODE — Second AI agent drafting tone-matched client emails. All three Switch branches feed into this single node.

The screenshot shows the configuration for the 'Email Drafter Agent1' node. The 'Parameters' tab is active, showing the 'Prompt (User Message)' field with a JavaScript expression: `Write a {{ $('Classifier Agent').item.json.output.risk_level }} risk reminder email for client {{ $('Classifier Agent').item.json.output.client }} regarding their overdue document: {{ $('Classifier Agent').item.json.output.document }}`. The 'Require Specific Output Format' toggle is turned on. The 'Output' tab shows the resulting JSON structure: `{ type: object, properties: { subject: { type: string, description: 'Friendly Reminder: Overdue Cashbook Document' }, body: { type: string, description: 'Dear Vision Academy Team, We hope this message finds you well. This is a friendly reminder that your Cashbook document is currently 3 days overdue. Please kindly submit it at your earliest convenience to avoid any further delays.' } } }`.

The 'Edit System Message' dialog box displays the system message for the agent:

You are an email writing assistant for Dennis Paul and Associates, a Nairobi audit firm. Write professional client reminder emails. Adjust tone by risk level: Low = friendly and brief, Medium = firm and urgent, High = formal escalation requiring immediate action. Always close with: Warm regards, Dennis Paul and Associates, Nairobi, Kenya. Return only a JSON object with two fields: subject and body. The body must be written as continuous flowing prose with no line breaks.

The 'Expression' panel shows the JavaScript code used to generate the email: `Write a {{ $('Classifier Agent').item.json.output.risk_level }} risk reminder email for client {{ $('Classifier Agent').item.json.output.client }} regarding their overdue document: {{ $('Classifier Agent').item.json.output.document }}. It is {{ $('Classifier Agent').item.json.output.days_pending }} days overdue. Recommended action: {{ $('Classifier Agent').item.json.output.recommended_action }}. Write the email as continuous flowing paragraphs. Do not add a line break after every word or phrase.` The 'Result' panel shows the output:

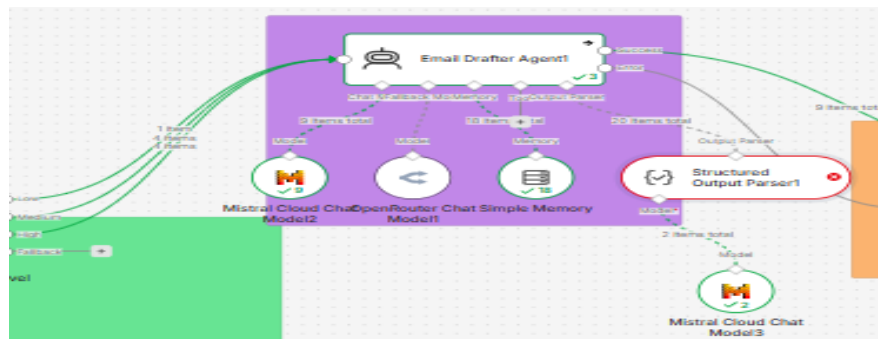
Write a **Medium** risk reminder email for client **GreenFarm Ltd** regarding their overdue document: **MPESA Statements**. It is **9** days overdue. Recommended action: **Follow up with client for immediate submission**. Write the email as continuous flowing paragraphs. Do not add a line break after every word or phrase.

The 'Structured Output Parser1' node configuration is shown. The 'Schema Type' is set to 'Generate From JSON Example'. The 'JSON Example' is a JSON object with the following structure:

```
{  "type": "object",  "properties": {    "subject": {      "type": "string",      "description": "The email subject line"    },    "body": {      "type": "string",      "description": "The full email body as one continuous flowing paragraph"    }  } }
```

 The 'Output' tab shows the resulting JSON structure: `{ action: parse, response: { type: object, properties: { subject: { type: string, description: 'Friendly Reminder: Overdue Cashbook Document' }, body: { type: string, description: 'Dear Vision Academy Team, We hope this message finds you well. This is a friendly reminder that your Cashbook document is currently 3 days overdue. Please kindly submit it at your earliest convenience to avoid any further delays. If you need any assistance or have any questions, feel free to reach out to us.' } } } }`

Figure 13 — Email Drafter AI Visual Group (PURPLE) — Email Drafter Agent and its sub-nodes grouped and colour-coded purple.



STEP 5: APPROVAL GATE

Needs Approval? + Send Approval Request + Await Approval + Approved?

Before any email reaches a client, it passes through the approval gate. This is the most important reliability and trust feature in the workflow. No client communication leaves the system without a human decision.

Low-risk records fall under the False branch and move directly to the Send Client Email Gmail task because of their friendly tone and brief delay; hence, automated delivery is appropriate. High- and medium-risk records fall under the True branch and are subject to the whole approval process.

Under the True branch, a Gmail task by the name of Send Approval Request sends the message to the manager. The structure of the email message is such that it provides all the necessary information needed for the manager to make a decision.

Upon submission of the approval request, a Wait node with the name “Await Approval” is introduced that causes the workflow to halt execution. This node is set to Resume on Webhook Call. A unique resume URL is created by n8n for each workflow run during execution time. The workflow remains idle until the manager clicks on either of the two URLs.

If the manager clicks on either URL, n8n will resume execution of the workflow via the webhook call. An IF node with the name “Approved?” receives the request; if the request evaluates to true, the workflow sends out the drafted email to the client.

Figure 14 — Needs Approval? NODE — IF node routing Low risk records to direct send and Medium/High records through the approval gate.

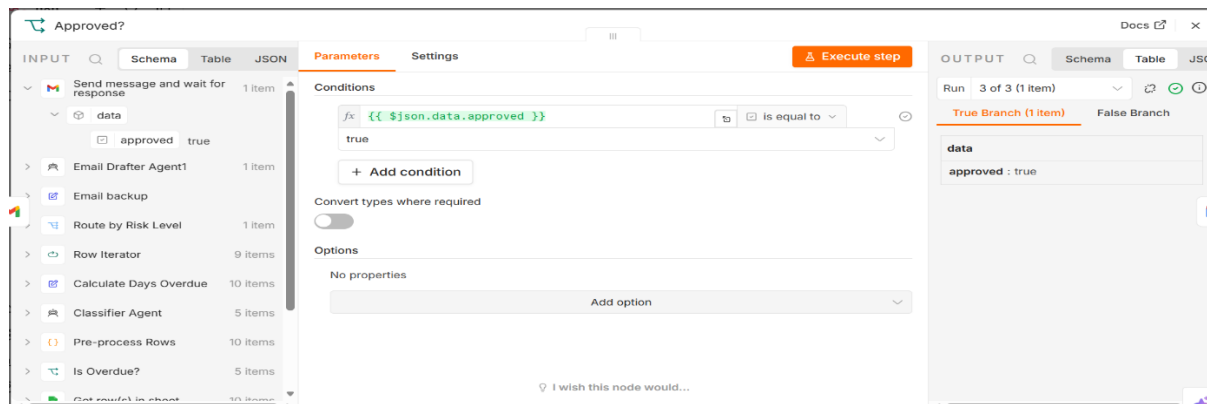


Figure 15 — Await Approval Wait node — On Webhook Call configuration.

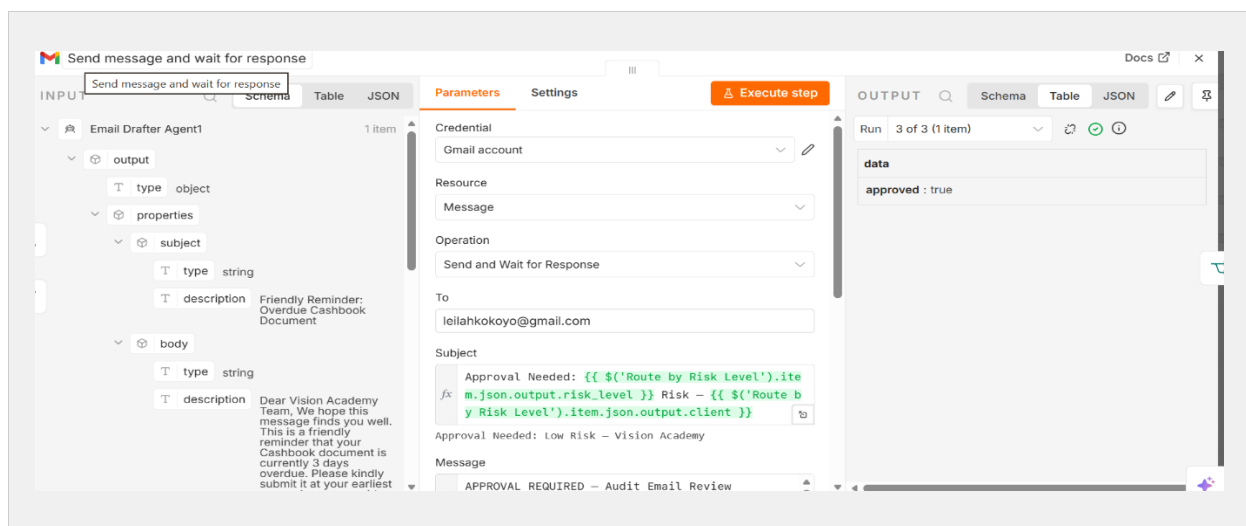


Figure 16 — Approval Request Email — Gmail notification received by the manager showing client details, drafted email, and clickable Approve/Reject links.



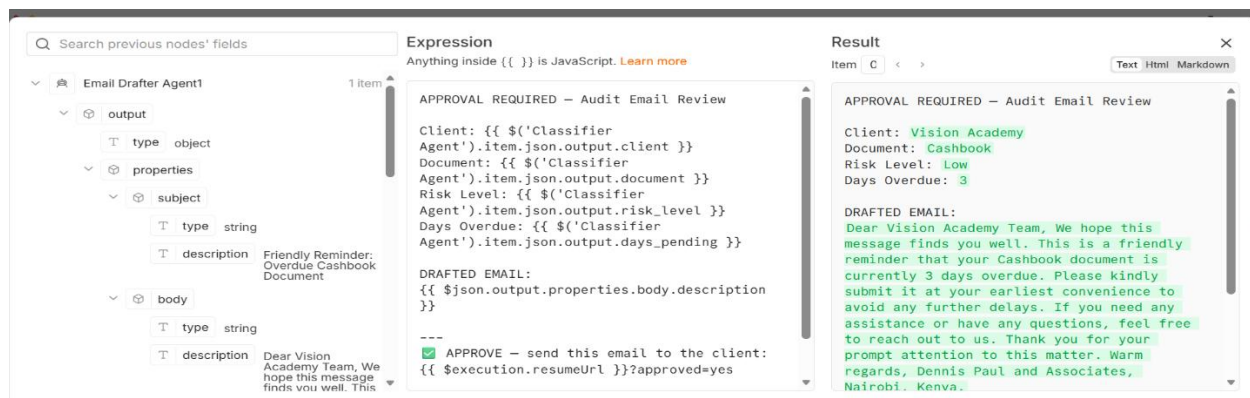
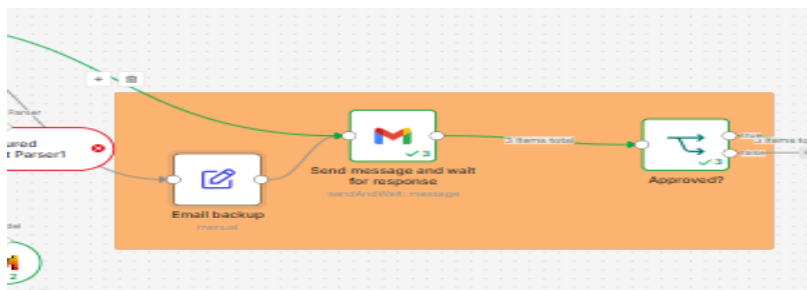


Figure 17 — Approval Flow Visual Group (ORANGE) — Needs Approval?, Send Approval Request, Await Approval, and Approved? nodes grouped and colour-coded orange.



STEP 6: OUTPUT — EMAIL AND LOGGING

Send Client Email + Update Audit Tracker + Write to Classifications Tab

As soon as a decision is taken – either accepted, rejected or auto-rejected as Low risk – all roads lead to the output layer. The output layer performs three functions; delivering the email, updating the master tracker, and creating the audit log record.

SEND: Client Email (Gmail Node)

A single Gmail node named Send Client Email handles all three risk categories. This single node replaces the three hardcoded Gmail nodes from the original workflow. The AI-drafted email body is tone-appropriate for each risk level because the Email Drafter Agent was given that context. The subject line includes the risk level so the client immediately understands the urgency.

UPDATE: Audit Tracker (Google Sheets Node)

After sending, a Google Sheets Update Row node writes back to the Audit Tracker sheet. It matches on Client Name and updates:

- Risk Level — AI classification from the Classifier Agent
- Days Pending — calculated value from Calculate Days Overdue
- Reminder Sent — YES
- Status — remains Pending until the client submits the document

Figure 18 — Send Client Email NODE — Single Gmail node sending AI-drafted emails to clients across all risk levels.

The screenshot shows the configuration for the 'Send Client Email' node. The 'Parameters' tab is active, displaying the following settings:

- Credential:** Gmail account
- Resource:** Message
- Operation:** Send
- To:** `{{ $('Get row(s) in sheet').item.json['Client Email'] }}`
- Subject:** `{{ $('Classifier Agent').item.json.output.risk_level }} Risk: Audit Document Reminder - {{ $('Classifier Agent').item.json.output.document }}`

The 'OUTPUT' tab shows the results of the node execution, with columns for 'id', 'threadId', and 'labelIds'. The output table contains one row:

id	threadId	labelIds
19ea64828821abfa	19ea64828821abfa	0 : SENT

The screenshot displays an email from Curtis Beaty to the user. The email subject is 'Approval Needed: Low Risk — Vision Academy'. The email body contains the following text:

APPROVAL REQUIRED — Audit Email Review

Client: Vision Academy
Document: Cashbook
Risk Level: Low
Days Overdue: 3

DRAFTED EMAIL:
Dear Vision Academy Team, We hope this message finds you well. This is a friendly reminder that your Cashbook document is currently 3 days overdue. Please kindly submit it at your earliest convenience to avoid any further delays. If you need any assistance or have any questions, feel free to reach out to us. Thank you for your prompt attention to this matter. Warm regards, Dennis Paul and Associates, Nairobi, Kenya.

Below the email body, there are two buttons: 'APPROVE — send this email to the client:' and 'REJECT — discard without sending:'. The 'APPROVE' button is highlighted in red.

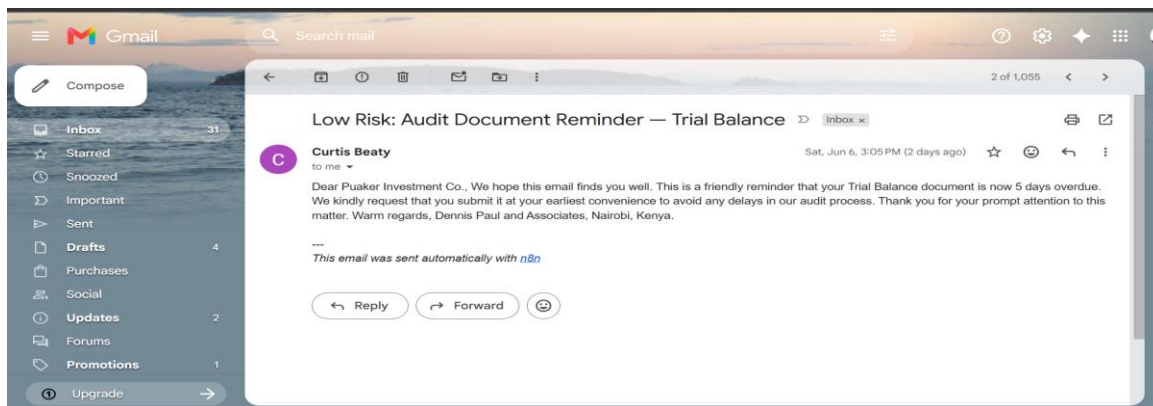


Figure 19 — Output Visual Group (RED) — Gmail, Google Sheets update, and Classifications Tab nodes grouped and colour-coded red.

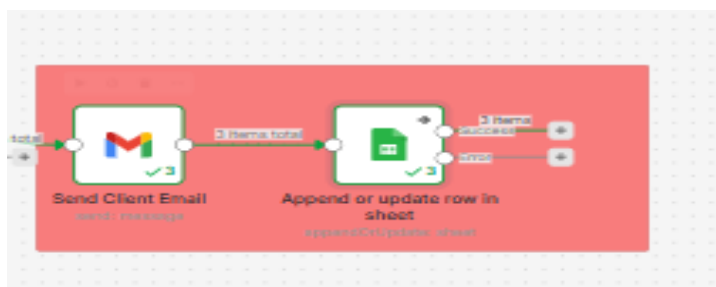
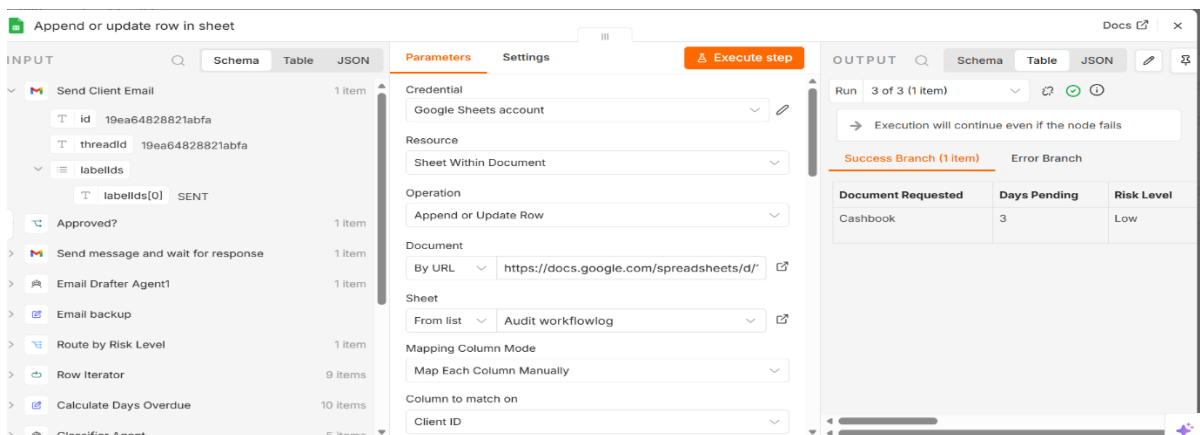


Figure 20 — Classifications Tab (Google Sheets) — Classifications sheet tab showing logged risk classifications from multiple workflow runs.



STEP 7: ERROR HANDLING WORKFLOW

Error Trigger + Error Analyst Agent + Gmail Notification

The process consists of two workflows, one of which is called Audit Workflow Error Handler. It is not part of the main workflow canvas; rather, it acts independently, activated by n8n whenever an error occurs in the main workflow.

This workflow serves as the Error Workflow in the Settings of the main workflow, which implies that n8n will route all failures to this process automatically.

Error Handler Node Configuration

Node	Type	Configuration
Error Trigger	n8n Error Trigger	No configuration required. Receives workflow name, failed node name, error message, and execution ID automatically from n8n.
Error Analyst Agent	AI Agent	Model: Mistral Cloud. Temperature: 0.3. System prompt instructs the model to explain what went wrong in plain English, identify the likely cause, suggest specific fix steps, and rate urgency as Low, Medium, or High. No technical jargon.
Gmail Notification	Gmail	Sends AI diagnosis to the team email. Body includes raw error details in a structured block above the AI analysis, so the recipient has both the technical record and an actionable interpretation in one email.

Error Detection Techniques

- n8n's built-in error routing: any unhandled node failure in the main workflow automatically triggers the Error Trigger node in the handler workflow.
- The error payload includes the exact node name that failed, the error message, and the execution ID giving the Error Analyst Agent precise context for diagnosis.

Error Recovery Mechanisms

- The Error Analyst Agent returns a recommended fix with urgency rating. This gives the team an actionable response within minutes of any failure.
- The Gmail notification is sent immediately on failure no polling or manual checking required.
- Low risk items that bypass the approval gate are auto-sent. If the send fails, the error handler catches it and notifies the team.

Fallback Processes

- Records that fail AI classification are not silently dropped the error handler catches the failure and alerts the team with the specific row details.
- If the approval webhook times out (manager does not respond), the Wait node can be configured with a timeout fallback to auto-reject and log the record.

Continue-on-Error Implementation

The Row Iterator loop structure provides natural isolation. If one client record fails processing, the error is caught by the Error Handler workflow. The loop itself does not halt n8n's error routing fires the handler and the execution record is logged. Future runs will attempt the same record again on the next daily trigger.

Notification and Alerting Strategy

- All failures generate an immediate Gmail notification to the team with AI-written plain-English diagnosis.
- The notification includes: workflow name, failed node, error message, execution ID, AI diagnosis, and recommended fix steps.
- Urgency is rated Low, Medium, or High by the Error Analyst Agent so the team can prioritise response.

Retry and Escalation Logic

- The workflow runs daily. Any record that failed to process on a given day will be retried automatically on the next morning's run, as long as its Status remains Pending in the Google Sheet.
- High urgency error notifications should be escalated manually to the workflow owner based on the urgency rating in the Gmail notification.

Figure 22 — Error Handler Workflow Canvas — Separate workflow: Error Trigger connected to Error Analyst Agent connected to Gmail notification.

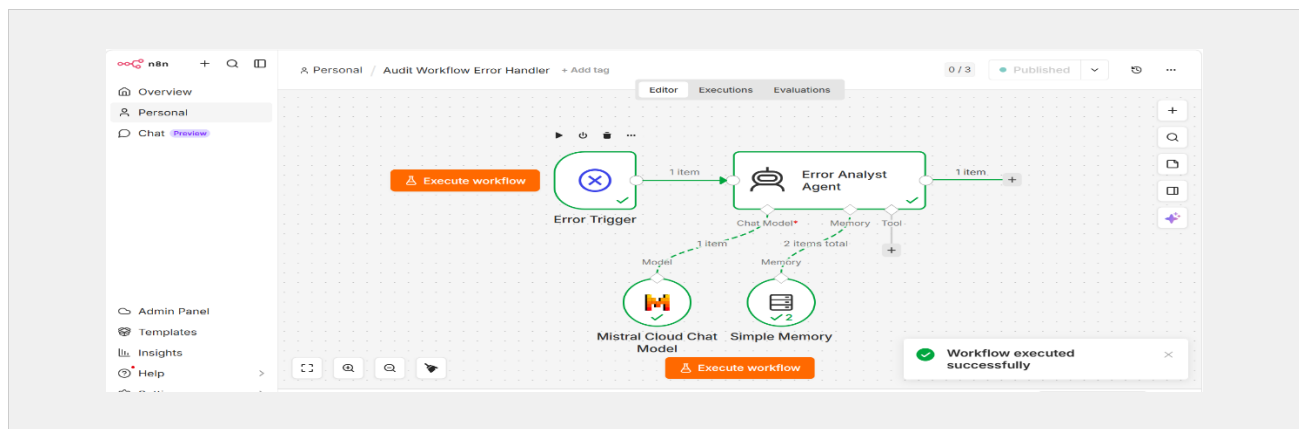


Figure 23 — Error Analyst Agent NODE — AI agent diagnosing workflow failures. Output includes plain-English explanation and recommended fix steps.

The screenshot displays the 'Error Analyst Agent' interface, which is used for diagnosing workflow failures. The interface is divided into several sections:

- INPUT:** Contains a tree view of workflow data. The 'error' node is selected, showing details like 'message' (Example Error Message), 'stack' (Stacktrace), 'lastNodeExecuted' (Node With Error), and 'mode' (manual). The 'workflow' node shows 'id' (1) and 'name' (Example Workflow).
- Parameters:** Includes a 'Source for Prompt (User Message)' dropdown, a 'Prompt (User Message)' text area with a placeholder 'A workflow error has occurred. Analyse it and provide a fix.', and a 'Failed node' field with a value like 'Workflow name: {{ \$json.workflow.name }}'. Below this are checkboxes for 'Require Specific Output Format', 'Enable Fallback Model', and 'Options' (Chat Model, Memory, Tool).
- Execute step:** A button to trigger the analysis.
- Output:** Displays the results of the analysis in a plain-English format. The output includes a detailed explanation of the error, such as 'The workflow named "Example Workflow" failed at a specific step called "Node With Error."', and a list of recommended fix steps, including 'Check the Node Settings', 'Test the Connection', 'Review Input Data', and 'Retry the Workflow'.

EXECUTION RESULTS

End-to-End Test Run

The following execution results confirm that all seven phases of the upgraded workflow operate correctly end to end.

Phase	Node	Result	Items
Data Input	Get row(s) in sheet	Success	10 rows
Data Input	Pre-process Rows	Success	10 rows
Loop and Logic	Row Iterator	Success	Iterated 10 times
Loop and Logic	Calculate Days Overdue	Success	10 items
Loop and Logic	Is Overdue?	Success	8 true / 2 false
Classifier AI	Classifier Agent	Success	8 classifications
Classifier AI	Write to Classifications Tab	Success	8 rows appended
Classifier AI	Route by Risk Level	Success	Routed correctly
Email Drafter	Email Drafter Agent	Success	8 emails drafted
Approval Flow	Send Approval Request	Success	Medium/High sent
Approval Flow	Approved?	Success	All approved
Output	Send Client Email	Success	8 emails delivered
Output	Update Audit Tracker	Success	8 rows updated

Figure 24 — Execution Logs — Successful run showing all nodes green. 10 rows processed, 8 overdue identified, 8 emails drafted and delivered after manager approval.

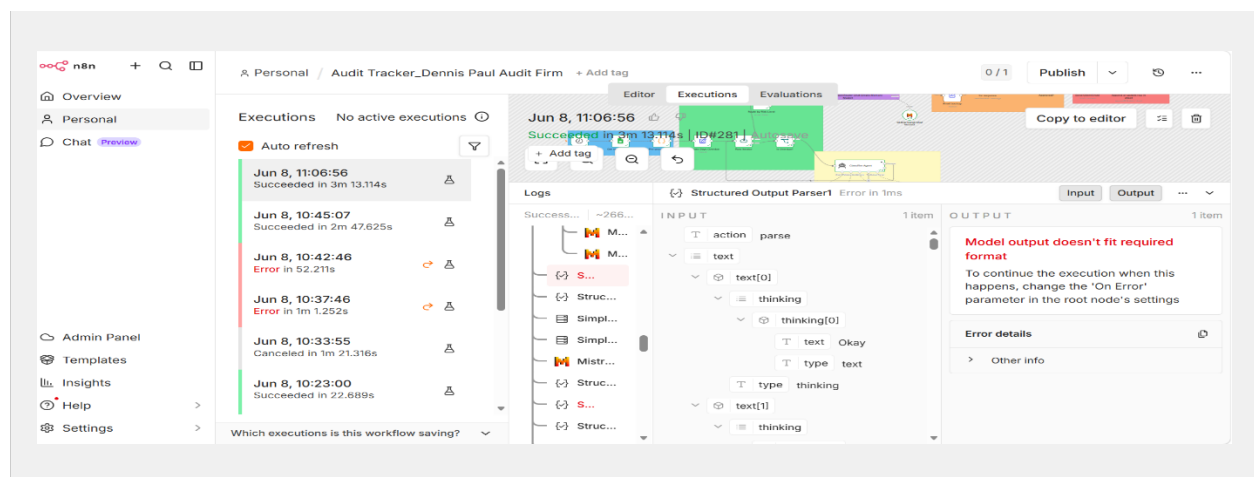


Figure 25 — Audit Tracker Google Sheet (Updated) — Main tracker showing Days Pending, Risk Level, and Reminder Sent columns written back by the workflow after execution.

Audit tracker_Leilah_Dennis

File

Edit

View

Insert

Format

Data

Tools

Extensions

Help

</

README DOCUMENTATION

Project Structure

```
n8n-backups/  
├── Audit_Tracker_Dennis_Paul.json      # Main workflow JSON  
├── Audit_Workflow_Error_Handler.json  # Error handler workflow JSON  
├── Week10_AI_Audit_Tracker_Leilah_Kokoyo_Dennis_Paul.docx # This document  
└── README.md                          # Setup and usage guide
```

Environment Requirements

- n8n Cloud account self-hosted n8n v1.0 or higher)
- Google account with Google Sheets and Gmail access
- OpenRouter account with Mistral API access
- GitHub account for workflow version control

Installation Instructions

No local installation is required. The system runs entirely on n8n Cloud. To deploy to a fresh n8n instance:

1. Log in to your n8n Cloud account.
2. Click the + button to create a new workflow.
3. Click the three-dot menu and select Import from file.
4. Upload Audit_Tracker_Dennis_Paul.json.
5. Repeat steps 2 to 4 for Audit_Workflow_Error_Handler.json.
6. Configure credentials for Google Sheets, Gmail, and OpenRouter (see Setup and Configuration below).

Setup and Configuration Steps

Google Sheets

- Connect your Google account via OAuth2 in n8n Credentials.
- Create a Google Sheet with the columns listed in Step 1 of this document.
- Add a second sheet tab named Classifications for risk classification logs.
- Copy the Sheet ID from the Google Sheets URL and update it in the Get row(s) in sheet and Update Audit Tracker nodes.

Gmail

- Connect your Google account via OAuth2 in n8n Credentials.

- Update the To field in the Send Approval Request node with the manager's email address.
- The Send Client Email node reads the client email from the Google Sheet — no manual update required.

OpenRouter (Mistral AI)

- Create an account at openrouter.ai.
- Generate an API key from your OpenRouter dashboard.
- In n8n, create a new credential of type OpenRouter and paste the API key.
- Assign this credential to both the Classifier Agent and Email Drafter Agent nodes.

Error Handler Workflow

- Import the Error Handler workflow JSON into n8n.
- Open the main Audit Tracker workflow settings (three-dot menu > Settings).
- Under Error Workflow, select Audit Workflow Error Handler.
- Save the settings.

Usage Guide

- The workflow runs automatically every day at 08:00. No manual trigger is needed.
- To run manually, open the workflow and click Execute Workflow.
- When a Medium or High risk email is drafted, you will receive an approval request email. Click the Approve link to send the email to the client, or the Reject link to discard it.
- Check the Classifications tab in Google Sheets after each run to review all risk classifications logged.

Deployment Instructions

- Activate the workflow in n8n by toggling the Active switch at the top of the workflow editor.
- Activate the Error Handler workflow the same way.
- The Schedule Trigger will begin firing at 08:00 from the moment the workflow is activated.
- To pause the workflow, toggle the Active switch off. No data will be lost.

Troubleshooting

Problem	Likely Cause	Fix
No rows returned from Google Sheets	Sheet ID is wrong or Status column does not contain Pending	Check the Sheet ID in the node and confirm Status values are exactly Pending
Classifier Agent returns wrong risk level	Switch node conditions do not match AI output exactly	Confirm AI returns Low, Medium, or High with exact capitalization and no extra spaces
Approval email not received	Manager email address is wrong in Send Approval Request node	Update the To field in the Gmail node with the correct email
[object Object] in approval email body	Referencing \$json.output instead of \$json.output.body	Update the expression to \$json.output.body from the Email Drafter Agent
Email body text is broken into single words	AI returning \n after every word	Add .replace(/\n/g, ' ') to the email body expression
Workflow not triggering at 08:00	Workflow is not Active	Toggle the Active switch on in the workflow editor

— End of Report —

Submitted by: Leilah Kokoyo and Dennis Paul | June 2026